

Coverage-Driven RTL Assertion Generation with Formal Exploration and Neuro-Symbolic Refinement

Zhiyuan Yan¹, Ziyue Zheng¹, and Hongce Zhang^{1,2}
 {zyan760, zzheng989}@connect.hkust-gz.edu.cn; hongcezh@hkust-gz.edu.cn

¹The Hong Kong University of Science and Technology (Guangzhou)

²The Hong Kong University of Science and Technology

ABSTRACT

Hardware functional verification relies on high-quality assertions to expose design bugs and establish confidence in Register Transfer Level (RTL) designs. Yet existing assertion mining methods still struggle to produce complete and reliable assertion sets: random or limited traces fail to cover hard-to-reach behaviors, and one-shot generation provides little feedback about what remains unverified or how the assertion set should be improved. As a result, critical design behaviors can remain uncovered even when many assertions are generated.

We present NeuroAssertion, a coverage-driven assertion generation framework that combines formal trace generation, syntax-guided synthesis (SyGuS), and an agent-inspired refinement process within a unified framework. Our framework first converts hard-to-reach control-flow conditions into formal reachability objectives, uses model checking to generate behaviorally diverse traces, and mines initial assertions from these traces with SyGuS. It then performs targeted agent-inspired refinement under verification feedback: one LLM first proposes candidate assertions for uncovered regions, and if a candidate fails formal checking, a second LLM generates a repair grammar that guides constrained symbolic synthesis in a neuro-symbolic repair procedure. Experimental results show that this framework delivers around 2× more assertions and about 2× higher mutation coverage than traditional assertion mining methods.

ACM Reference Format:

Zhiyuan Yan, Ziyue Zheng, and Hongce Zhang. 2026. Coverage-Driven RTL Assertion Generation with Formal Exploration and Neuro-Symbolic Refinement. In *2026 ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD '26)*, September 07–09, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3831599.3840311>

1 INTRODUCTION

Hardware functional verification increasingly depends on high-quality assertions to expose bugs and establish confidence in Register Transfer Level (RTL) designs. A central technique is assertion-based verification (ABV), where SystemVerilog Assertions (SVAs) capture expected design behaviors for simulation and formal property verification. Yet writing high-quality SVAs remains labor-intensive and error-prone, making assertion construction a persistent bottleneck in modern verification flows.

To reduce this manual effort, a substantial body of prior work has studied mining assertions from simulation traces. GoldMine [18] mines assertions from RTL simulation traces using static analysis and decision-tree learning. Liu et al. [9, 10] extended this direction from transaction-level models to word-level feature discovery, enabling mined assertions to capture relationships between multi-bit signals. Sheridan et al. [16] further emphasized coverage-aware assertion selection to reduce redundancy. Later template-based methods, including A-TEAM [4] and HARM [7], imposed stronger structural priors on the mining process. Most recently, SMART [22] combined SyGuS with oracle-guided counterexample refinement to improve trace-driven assertion synthesis.

Despite this progress, existing workflows still face two limitations. First, mining quality is constrained by trace diversity: when initial mining relies on random or constrained-random traces, hard-to-reach RTL behaviors often remain unobserved, leaving the initial assertion set incomplete. Second, existing methods provide little feedback about assertion completeness. They mine assertions from observed traces without explicitly identifying uncovered behaviors or how refinement should proceed. As a result, assertion mining remains largely one-shot: missed behaviors stay hidden, and improvement lacks direction. In practice, however, engineers use coverage feedback to extend the assertion set toward uncovered behaviors.

These limitations suggest that RTL assertion mining should be treated as a coverage-driven refinement problem rather than a one-shot generation problem. NeuroAssertion follows this view. It first uses formal exploration to convert hard-to-reach control-flow conditions into reachability objectives and generate traces that expose harder-to-reach behaviors before SyGuS-based mining. It then uses mutation-coverage feedback to identify uncovered obligations, generate LLM-based candidate assertions, and repair invalid ones through an agent-inspired loop with grammar-constrained symbolic synthesis. Together, these stages expand the behavioral basis of mining and refine the assertion set under explicit mutation-coverage feedback and formal checking.

Overall, this paper makes the following contributions:

- We propose NeuroAssertion, a coverage-driven assertion generation framework that combines formal exploration, syntax-guided synthesis, and an agent-inspired refinement loop within a unified framework, enabling assertion mining to move beyond one-shot generation over limited observed behaviors.
- We introduce an agent-inspired refinement process that uses mutation-coverage feedback to target uncovered obligations with LLM-generated candidate assertions and then repairs failed candidates through LLM-generated grammars for constrained symbolic synthesis.

- We demonstrate on seven RTL benchmarks that the full framework consistently improves both assertion quantity and mutation coverage over SMART and direct LLM generation.

2 PRELIMINARIES

We consider RTL designs written in Verilog/SystemVerilog, with target properties expressed as SystemVerilog Assertions (SVAs). This section reviews the two ingredients underlying our methodology: SyGuS-based assertion mining and mutation-coverage feedback.

2.1 SyGuS-Based Assertion Mining

Syntax-Guided Synthesis (SyGuS) synthesizes programs that satisfy a specification under a context-free grammar [1]. Given behavioral constraints φ and a grammar $G = (NT, T, S, R)$, where NT is the set of nonterminals, T is the set of terminals, S is the start symbol, and R is the set of production rules, SyGuS searches for formula P derivable from G such that P satisfies φ . By restricting candidates to those generated by G , SyGuS turns unconstrained synthesis into structured search over a grammar-defined space.

Prior work [22] shows that this formulation is effective for RTL assertion mining: traces provide the behavioral constraints, while the grammar specifies the allowed operators, signal combinations, and temporal structures of the synthesized SVAs. To keep search tractable, existing methods partition RTL variables into smaller groups, assign a predefined grammar to each group, and synthesize assertions over each local signal set. Counterexamples from failed assertions are then fed back through a counterexample-guided abstraction refinement (CEGAR) loop to refine the trace-derived constraints. Our work inherits this SyGuS-centered formulation, but strengthens the behavioral basis of mining through formal exploration and extends refinement beyond the initial SyGuS loop.

2.2 Mutation Coverage and Uncovered Obligations

In our framework, mutation coverage serves not only as an evaluation metric but also as a refinement signal. We inject mutants that perturb RTL operators, conditions, or assignments, and measure whether the current assertion set can distinguish them from the original design through formal checking. Mutants that remain undistinguished expose behaviors insufficiently constrained by the current assertions. We treat the resulting missing constraints as *uncovered obligations*: explicit verification targets that connect coverage to refinement. Rather than treating incompleteness as an abstract problem, we localize it into specific uncovered behaviors that guide downstream generation and repair. In our methodology, uncovered obligations trigger targeted neural proposal, while formal checking determines whether a candidate can be accepted directly or must be repaired symbolically.

3 METHODOLOGY

3.1 Challenges to Address

3.1.1 C1: Limited Behavioral Reachability. Despite improvements in synthesis and template design, most assertion mining workflows still rely on random or constrained-random traces as their initial

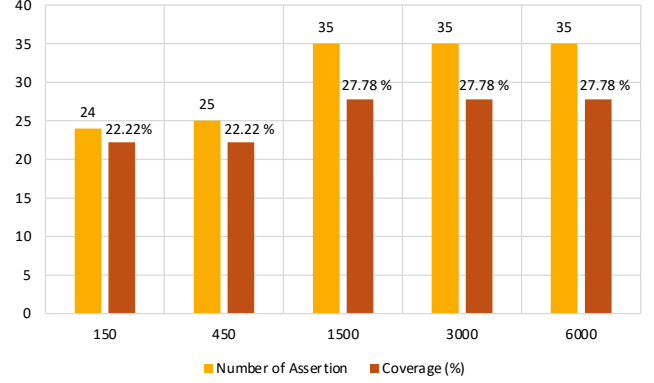


Figure 1: Coverage and assertion saturation with increasing random test cases on the b12 benchmark. Despite a 40× increase in simulation effort, branch coverage plateaus at about 27% and the assertion count saturates at 35.

behavioral basis. This makes mining fundamentally reachability-limited: if important RTL behaviors are never exercised, no downstream mining step can recover them. Hard-to-reach conditions are especially problematic. For example, a branch condition such as $a == 8'd222$ over an 8-bit signal is triggered with probability only $1/256$ in each random test.

More importantly, simply increasing the number of random simulations does not remove this bottleneck. As shown in Fig. 1, even when using the state-of-the-art SMART method [22] and scaling from 150 to 6000 test cases on the b12 benchmark from the ITC'99 suite [5], both branch coverage and the number of mined assertions plateau at around 28% and 35, respectively. This saturation indicates that random exploration reaches a ceiling on certain designs, leaving potentially important RTL behaviors unobserved and limiting the completeness of the mined assertion set.

3.1.2 C2: Weak Refinement Feedback. Even after an initial assertion set has been mined, existing workflows provide limited guidance about how the assertion set should be expanded. They mainly validate whether the current assertions hold, but seldom identify which design behaviors remain uncovered or elevate these gaps into explicit refinement targets for the next iteration. As a result, the available feedback is largely about the correctness of existing assertions rather than the completeness of the assertion set. This stands in contrast to practical verification workflows, where engineers often inspect coverage feedback to decide what assertions should be added next. Existing automated methods lack a similarly directed refinement signal.

3.2 Overview of NeuroAssertion

The above challenges suggest treating RTL assertion mining as a coverage-driven refinement loop rather than a one-shot generation process. To address C1, NeuroAssertion introduces S1, a formal exploration stage that converts hard-to-reach control-flow conditions into reachability objectives and uses the resulting traces to strengthen SyGuS-based initial mining. To address C2, it introduces S2, a feedback-driven neuro-symbolic refinement stage that uses

mutation coverage to identify uncovered obligations, propose candidate assertions, and repair candidates that fail checking through grammar-constrained symbolic synthesis.

Fig. 2 summarizes the pipeline. RTL instrumentation and model checking first generate traces that strengthen SyGuS-based mining and produce an initial assertion set $\mathcal{A}_0$. Mutation analysis then identifies uncovered obligations and drives a refinement cycle of neural proposal, formal checking, and symbolic repair. Candidates that pass checking are added directly to the assertion set, while candidates that fail are repaired symbolically. In this way, mutation analysis provides completeness feedback, and formal verification provides correctness feedback for targeted refinement.

3.3 S1: Formal Exploration for Enhanced Assertion Mining

To address C1, S1 strengthens assertion mining before refinement begins. The key idea is to expand the behavioral basis of mining by actively seeking executions that random simulation is unlikely to expose. Prior work has shown that formal reachability analysis can systematically drive execution toward target behaviors that are hard to exercise through random simulation [25]. Following this intuition, we first perform RTL instrumentation for targeted reachability analysis, converting target control-flow conditions into explicit formal reachability objectives. We then use model checking to search for executions that satisfy these objectives. In this way, the generated traces expose a broader range of execution patterns and provide a stronger basis for downstream assertion synthesis.

Fig. 3 shows this instrumentation process. For each branch condition in the RTL, we introduce zero-initialized auxiliary coverage registers `br_cov` whose bits record whether the corresponding branches have been exercised, and we insert update statements that set the relevant bit when the target branch is taken. We then generate SVAs over these coverage bits as formal objectives. Counterexample traces returned by the model checker become test vectors that exercise the target branches.

After obtaining these traces, we run SyGuS within an iterative CEGAR loop to synthesize an initial assertion set $\mathcal{A}_0$ over the observed behaviors. Counterexamples returned by formal verification are used to refine the trace-derived constraints and guide SyGuS toward improved assertions. S1 therefore addresses C1 not by changing the symbolic miner itself, but by giving it a richer set of behaviors to mine from.

3.4 S2: Coverage-Driven Neuro-Symbolic Refinement

To address C2, S2 uses mutation coverage to derive a set of uncovered obligations $\mathcal{U}_0$ from the initial assertion set $\mathcal{A}_0$ and uses them to drive refinement through two tightly coupled stages: an obligation-driven neural proposal stage and a grammar-constrained symbolic repair stage.

3.4.1 Obligation-Driven Neural Proposal. Given RTL context and uncovered obligations $\mathcal{U}_0$, the first LLM generates a candidate set $\tilde{\mathcal{A}}$. The role of this stage is not to certify correctness, but to provide semantically informed proposals for regions that remain uncovered after the initial symbolic mining stage. In this sense, it serves as

the neural front end of the refinement loop: the LLM expands the candidate space for hard-to-cover behaviors, while correctness is delegated to the formal validation stages that follow.

To keep the generated candidates useful for downstream verification, we use a structured prompting skill that guides the model through four steps:

- **Mutation localization and context extraction:** Identify the mutation site, the changed expression, and the surrounding RTL context.
- **Circuit type detection:** Determine whether the design is combinational or sequential, and identify the relevant clock and reset signals.
- **Behavioral intent inference:** Infer how the uncovered obligation changes the intended behavior and express that behavior as logical conditions.
- **Signal extraction and assertion synthesis:** Extract the relevant signals and emit synthesis-compatible SystemVerilog assertions with metadata.

Each candidate is emitted as a structured JSON object containing the assertion, its semantic description, the required signals, and verification metadata. The prompt constrains outputs to synthesis-compatible SystemVerilog assertions that can be checked by the formal backend without manual rewriting. As a result, the proposal stage integrates smoothly with the later verification and repair stages instead of introducing an additional manual translation step.

3.4.2 Grammar-Constrained Symbolic Repair. Each candidate assertion proposed in the previous stage is checked immediately by the formal backend. Valid assertions are inserted directly into the current assertion set, while invalid ones are passed to a repair stage rather than being discarded. The main reason is that the variables involved in a failed candidate often still encode meaningful behavioral relationships, even if the assertion itself is not yet correct. Restarting generation from scratch would discard this useful structure.

For a failed candidate assertion $\tilde{a} \in \tilde{\mathcal{A}}_t$, we extract its relevant signal set S and use the second LLM to map the RTL context R and S to a repair-oriented grammar:

$$G = \text{LLM}_2(R, S)$$

This constrained grammar turns repair from an unconstrained symbolic search under a general grammar into a local search centered on the signals implicated by the failed candidate. The neural component therefore does not emit a repaired assertion directly; instead, it defines a semantically focused symbolic search space.

Once this grammar is constructed, the remaining repair process is symbolic. We invoke SyGuS under the grammar G and the constraint set C to synthesize a repaired assertion:

$$a^* = \text{SyGuS}(G, C)$$

If a^* still fails formal validation, we preserve the newly observed counterexample and continue repair under strengthened constraints. When the LLM-induced grammar is insufficient, we fall back to a default grammar over the same signal set and continue counterexample-guided symbolic synthesis.

Algorithm 1 summarizes the overall refinement procedure. For each uncovered obligation, the first LLM proposes a candidate assertion (line 2), and the formal backend checks it immediately

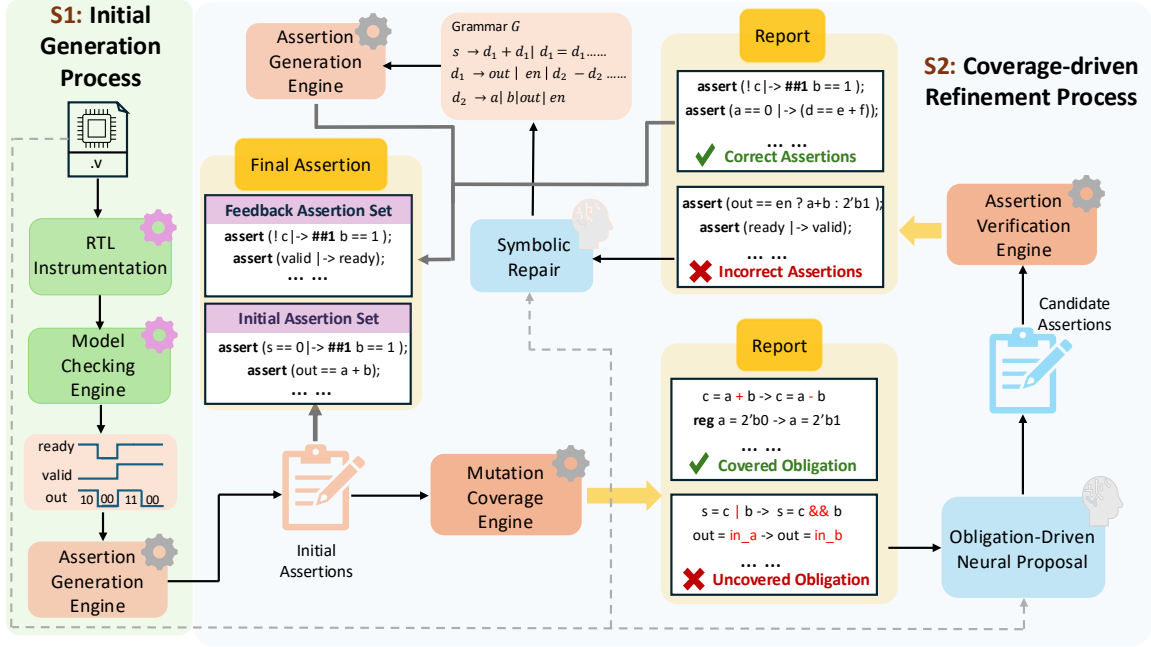


Figure 2: Overview of NeuroAssertion. S1 performs initial assertion generation, while S2 performs feedback-driven refinement. The final assertion set combines initial, directly verified, and repaired assertions.

```

module top(input clk, input in);
reg [6:0] state;
reg [999:0] br_cov = 1000'b0;
always @(posedge clk) begin
  case (state)
    7'b000000:begin
      br_cov[b0] = 1'b1;
      if (in) begin
        state = 7'b0010000;
        br_cov[b1] = 1'b1;
      end else begin
        state = 7'b0010010;
        br_cov[b2] = 1'b1;
      end end
    7'b0010000:begin
      br_cov[b3] = 1'b1;
    end endcase end
    .....
  assert property(br_cov[b0] == 1'b0);
endmodule

```

Figure 3: RTL instrumentation for targeted reachability analysis. Inserted code is color-coded: auxiliary registers and branch markers are highlighted in green, while the generated SVA is highlighted in red.

(line 3). Candidates that already satisfy formal checking are inserted directly into the assertion set (line 5).

If the initial candidate fails, the returned counterexample is added to the trace-derived constraints from S1 to initialize the repair constraint set (line 8), and the relevant signals are extracted from the failed candidate (line 9). The second LLM then generates a repair grammar (line 10). The grammar-guided repair solver runs counterexample-guided symbolic synthesis under that grammar and returns either a valid repaired assertion or a new counterexample (line 11). If repair succeeds, the repaired assertion is inserted into the assertion set (line 13). Otherwise, the new counterexample is added to the constraint set (line 16), the search switches to a default grammar over the same signal set (line 17), and a fallback

Algorithm 1: Coverage-driven refinement with neural proposal and symbolic repair

Input: RTL context R , trace-derived constraint set C_{trace} , current assertion set $\mathcal{A}$, uncovered obligations $\mathcal{U}$

Output: Refined assertion set $\mathcal{A}$

```

1  foreach  $u \in \mathcal{U}$  do
2     $\tilde{a} \leftarrow \text{LLM}_1(R, u)$ ;
3     $c \leftarrow \text{FormalCheck}(R, \tilde{a})$ ;
4    if  $c = \emptyset$  then
5       $\mathcal{A} \leftarrow \mathcal{A} \cup \{\tilde{a}\}$ ;
6      continue;
7    end
8     $C \leftarrow C_{\text{trace}} \cup \{c\}$ ;
9     $S \leftarrow \text{ExtractSignals}(\tilde{a})$ ;
10    $G \leftarrow \text{LLM}_2(R, S)$ ;
11    $a^*, c \leftarrow \text{GrammarGuidedRepair}(R, G, C)$ ;
12   if  $c = \emptyset$  then
13      $\mathcal{A} \leftarrow \mathcal{A} \cup \{a^*\}$ ;
14     continue;
15   end
16    $C \leftarrow C \cup \{c\}$ ;
17    $G \leftarrow \text{DefaultGrammar}(S)$ ;
18    $a^* \leftarrow \text{DefaultRepair}(R, G, C)$ ;
19    $\mathcal{A} \leftarrow \mathcal{A} \cup \{a^*\}$ ;
20 end

```

symbolic repair solver is invoked (line 18). The resulting valid assertion is then added to the assertion set (line 19). In this way, the neural stages define the repair space and the symbolic stages carry out counterexample-guided synthesis and final certification.

4 EXPERIMENT

4.1 Experimental Setup

The experiments are conducted on a machine with a 2.9 GHz Intel Xeon Platinum 8375C CPU and 256 GB RAM. For the initial assertion-mining stage, we follow the variable-grouping heuristic

Table 1: The statistics of benchmark designs in the experiment.

Design	# RTL Lines	# Variables
Arb2	59	13
B12	738	26
I2C	1114	155
Ibex_controller	1101	233
Ibex_decoder	1156	141
Multdiv	353	59
Pico	2530	759

of [22], and we use cvc5 [3] as the SyGuS solver for assertion synthesis. For all LLM-related tasks in our framework, including assertion proposal and grammar generation, we use GPT-5 as the underlying large language model. For formal checking, we use SymbiYosys [24] to preprocess the design and translate it into the BTOR2 format, and then use Pono [13] to verify whether the generated assertions hold on the design.

As the main baseline, we compare against SMART, the oracle-guided SyGuS-based method proposed by Ye et al. [22]. We choose SMART because it is the most closely related prior work to our approach, sharing the same SyGuS-centered assertion generation framework. In contrast, our method further introduces formal exploration and an LLM-guided refinement stage after the initial mining stage. Earlier methods such as GoldMine [18] and HARM [7] represent important prior approaches, but SMART already showed stronger results than these baselines, making it the most appropriate primary comparison target in our experiments.

4.2 Benchmarks

We evaluate our method on seven RTL benchmarks spanning a range of design sizes and complexities. Arb2 is a compact arbitration benchmark adopted from prior assertion-mining evaluation [22], and B12 is taken from the ITC'99 benchmark suite [5]. I2C corresponds to the OpenCores I2C controller [8]. To include processor-related RTL modules, we also evaluate three components from the open-source Ibex RISC-V core [11]: Ibex_controller, Ibex_decoder, and Multdiv. Finally, Pico is derived from the PicoRV32 RISC-V core [23]. Table 1 summarizes the statistics of these benchmarks. Overall, the benchmark set covers designs ranging from 59 to 2530 RTL lines and from 13 to 759 variables, providing a heterogeneous testbed for evaluating assertion generation across substantially different design sizes.

4.3 Experimental Result

Figure 4 presents the overall experimental results of our method and SMART [22] on the benchmark suite. Overall, our method achieves stronger performance across the benchmarks, indicating that the proposed coverage-driven refinement pipeline improves the quality of the generated assertion set beyond a purely SyGuS-based workflow. In terms of assertion quantity, our method generally produces around twice as many assertions as SMART. The gain is especially pronounced on larger designs such as I2C (89 vs. 244), Ibex_controller (136 vs. 297), Ibex_decoder (104 vs. 246), and Pico (358 vs. 477). In terms of mutation coverage, our method

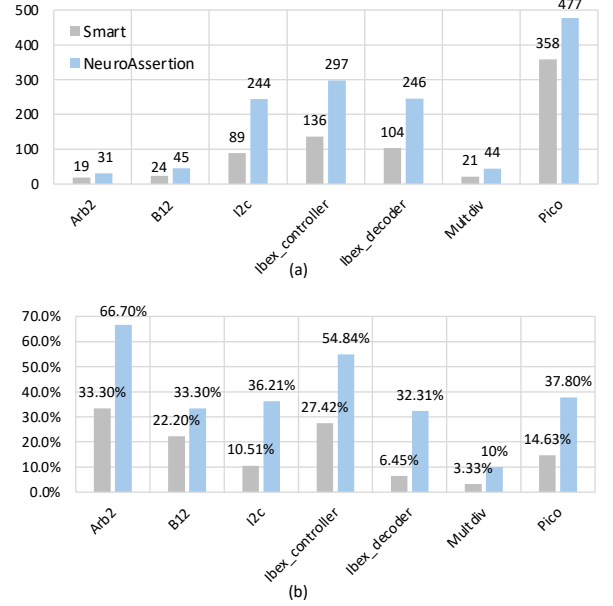


Figure 4: Overall experimental results compared with SMART. (a) shows the number of generated assertions, and (b) shows mutation coverage rate.

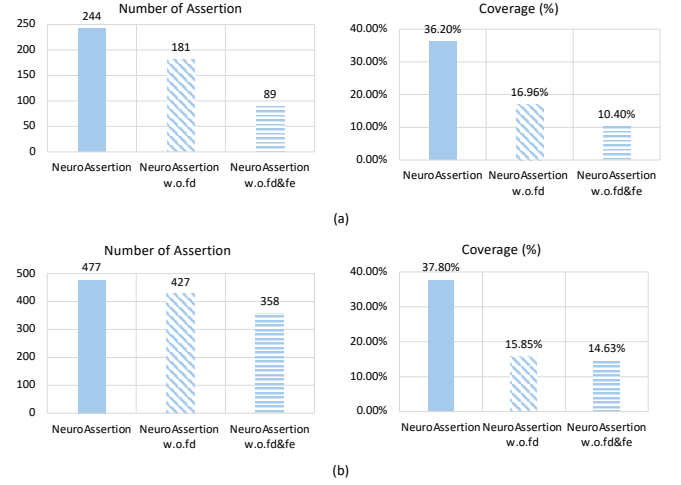


Figure 5: Ablation study of NeuroAssertion on representative benchmarks. (a) I2C, and (b) Pico.

also delivers around a twofold improvement overall, and on several benchmarks the gain is even larger, including I2C (10.51% to 36.21%), Ibex_decoder (6.45% to 32.31%), and Pico (14.63% to 37.80%). These results show that our method not only generates more assertions, but also produces assertion sets that detect substantially more mutations.

4.4 Ablation Study

Figure 5 further studies the contribution of the formal exploration and feedback components. We compare the full NeuroAssertion framework with two reduced variants: NeuroAssertion w.o.fd and NeuroAssertion w.o.fd&fe, which remove the feedback component

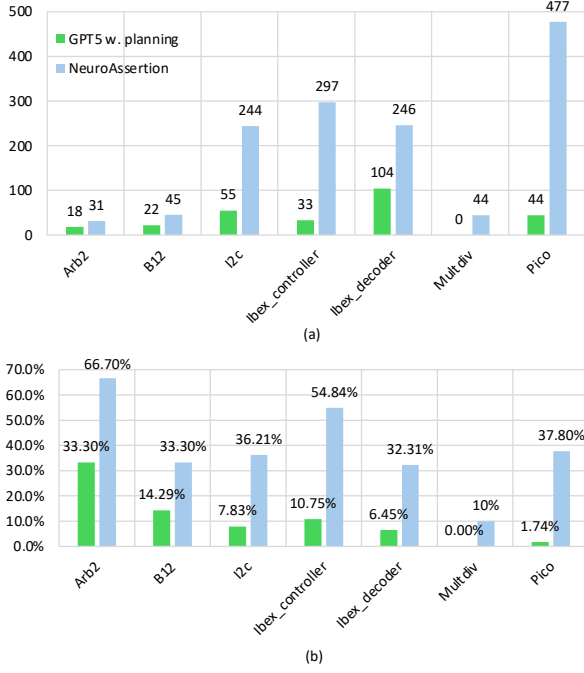


Figure 6: Comparison with a direct LLM-based baseline. Subfigure (a) shows the number of generated assertions, and subfigure (b) shows mutation coverage.

(fd) and both the formal-exploration and feedback components (fd&fe), respectively. We report results on two representative benchmarks, I2C and Pico.

The ablation results show that both components are important to the final performance. On I2C, the full method generates 244 assertions and achieves 36.20% mutation coverage, while NeuroAssertion w.o.fd drops to 181 assertions and 16.96% coverage, and NeuroAssertion w.o.fd&fe further drops to 89 assertions and 10.40% coverage. A similar trend appears on Pico, where the full method generates 477 assertions with 37.80% coverage, compared with 427 assertions and 15.85% coverage for NeuroAssertion w.o.fd, and 358 assertions and 14.63% coverage for NeuroAssertion w.o.fd&fe.

These results indicate that formal exploration is essential for exposing richer design behaviors and increasing the number of useful candidate assertions, while the refinement process is critical for turning these candidates into assertions that improve mutation coverage. Together, these components make the full NeuroAssertion framework substantially more effective than its reduced variants.

4.5 Comparison with Direct LLM Generation

Existing assertion-mining methods,¹ including our main comparison with SMART, do not directly evaluate whether an LLM alone can generate assertions from RTL code. To study this setting, we construct a direct LLM baseline, denoted GPT-5 w. planning. This baseline uses GPT-5 with task-specific prompts that first analyze the RTL structure and then generate assertions for connectivity, register behavior, function, constraints, and safety. In this way, the baseline

¹We exclude other RTL-oriented methods because they target different settings: Orenes-Vera et al. [15] involves iterative manual prompt engineering, while Yan and Zhang [21] focuses on accelerating model checking rather than automatic assertion generation.

still benefits from prompt-level decomposition and planning, but it does not use our formal exploration stage or feedback-driven neuro-symbolic refinement loop.

Figure 6 shows that NeuroAssertion substantially outperforms this direct LLM baseline on both assertion quantity and mutation coverage. In terms of assertion quantity, NeuroAssertion generally produces several times more assertions on complex benchmarks, including I2C (244 vs. 55), Ibex_controller (297 vs. 33), Pico (477 vs. 44), and Multdiv (44 vs. 0). In terms of mutation coverage, the gap is even larger: NeuroAssertion improves coverage from 7.83% to 36.21% on I2C, from 10.75% to 54.84% on Ibex_controller, from 1.74% to 37.80% on Pico, and from 0.00% to 10.00% on Multdiv. Even on benchmarks where GPT-5 w. planning can generate a nontrivial number of assertions, such as Arb2 and Ibex_decoder, NeuroAssertion still achieves much higher coverage.

These results show that prompt-level planning alone is insufficient for high-quality RTL assertion generation. Although GPT-5 w. planning can produce some plausible assertions directly from RTL structure, without formal exploration it misses many hard-to-reach behaviors, and without our refinement loop it cannot reliably repair weak or incorrect candidates. NeuroAssertion therefore provides a much more effective way to turn LLM reasoning into assertions that are both numerous and verification-useful.

5 RELATED WORKS

Recent machine-learning-based methods for hardware verification can be broadly grouped into methods driven primarily by specification-side artifacts and RTL-oriented methods. Methods in the former category are driven by natural-language requirements, waveform descriptions, or coverage targets, including NL2SVA [17], ChatSVA [6], ChIRAAG [12], AssertLLM [20], CoverAssert [19], and LASP [2]. Representative RTL-oriented methods include AutoSVA [14], the RTL-focused LLM-assisted flow of Orenes-Vera et al. [15], and the trace-based word-level assertion generation method DreamMiner [21]. However, the former still involves iterative manual prompt engineering, while the latter focuses on accelerating model checking rather than automatic assertion generation. Our method belongs to the RTL-oriented category, but its focus is to automatically refine assertions through a feedback-driven loop that combines LLM-based generation with traditional assertion-mining and formal-checking tools under RTL-grounded mutation coverage.

6 CONCLUSION

This paper presented NeuroAssertion, a coverage-driven framework for RTL assertion generation that combines formal exploration, syntax-guided synthesis, and a feedback-driven neuro-symbolic refinement process. NeuroAssertion first expands the behavioral basis of assertion mining through RTL instrumentation and model checking, and then uses mutation-coverage feedback to identify uncovered obligations that guide targeted neural proposal and symbolic repair. In this way, the framework turns mutation analysis from a post hoc evaluation metric into an explicit refinement signal for assertion generation. Experiments further show that NeuroAssertion consistently outperforms SMART and a direct LLM baseline, producing around 2× more assertions and achieving about 2× higher mutation coverage overall.

REFERENCES

- [1] Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo MK Martin, Mukund Raghothaman, Sanjit A Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. *Syntax-guided synthesis*. IEEE.
- [2] Avinash Ayalasomayajula, Rui Guo, Jingbo Zhou, Sujun Kumar Saha, and Farimah Farahmandi. 2024. LASP: LLM Assisted Security Property Generation for SoC Verification. In *2024 ACM/IEEE 6th Workshop on Machine Learning for CAD (MLCAD)*. 29:1–29:7. <https://doi.org/10.1145/3670474.3685967>
- [3] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Alex Ozdemir, Andres Nötzli, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. 2022. cvc5: A versatile and industrial-strength SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 415–442.
- [4] Alessandro Danese, Nicolò Dalla Riva, and Graziano Pravadelli. 2017. A-team: Automatic template-based assertion miner. In *Proceedings of the 54th Annual Design Automation Conference 2017*. 1–6.
- [5] Scott Davidson. 1999. Characteristics of the ITC'99 benchmark circuits. In *IEEE International Test Synthesis Workshop (ITSW)*. 87.
- [6] Lik Tung Fu, Jie Zhou, Shaokai Ren, Mengli Zhang, Jia Xiong, Hugo Jiang, Nan Guan, Xi Wang, and Jun Yang. 2026. ChatSVA: Bridging SVA Generation for Hardware Verification via Task-Specific LLMs. *arXiv preprint arXiv:2604.02811* (2026).
- [7] Samuele Germiniani and Graziano Pravadelli. 2022. Harm: a hint-based assertion miner. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 4277–4288.
- [8] Richard Herveille. 2026. OpenCores I2C Controller Core. <https://opencores.org/projects/i2c>. Accessed: 2026-04-12.
- [9] Lingyi Liu, Chen-Hsuan Lin, and Shobha Vasudevan. 2012. Word level feature discovery to enhance quality of assertion mining. In *Proceedings of the International Conference on Computer-Aided Design*. 210–217.
- [10] Lingyi Liu, David Sheridan, Viraj Athavale, and Shobha Vasudevan. 2011. Automatic generation of assertions from system level design using data mining. In *Ninth ACM/IEEE International Conference on Formal Methods and Models for Codesign (MEMPCODE2011)*. IEEE, 191–200.
- [11] lowRISC. 2026. Ibex RISC-V Core. <https://github.com/lowRISC/ibex>. Accessed: 2026-04-14.
- [12] Bhabesh Mali, Karthik Maddala, Sweeya Reddy, Vatsal Gupta, Chandan Karfa, and Ramesh Karri. 2024. ChIRAAG: ChatGPT Informed Rapid and Automated Assertion Generation. *arXiv preprint arXiv:2402.00093* (2024).
- [13] Makai Mann, Ahmed Irfan, Florian Lonsing, Yahan Yang, Hongce Zhang, Kristopher Brown, Aarti Gupta, and Clark Barrett. 2021. Pono: a flexible and extensible SMT-based model checker. In *Computer Aided Verification: 33rd International Conference, CAV 2021, Virtual Event, July 20–23, 2021, Proceedings, Part II 33*. Springer, 461–474.
- [14] Marcelo Orenes-Vera, Aninda Manocha, David Wentzlaff, and Margaret Martonosi. 2021. AutoSVA: Democratizing formal verification of RTL module interactions. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 535–540.
- [15] Marcelo Orenes-Vera, Margaret Martonosi, and David Wentzlaff. 2023. Using Large Language Models to Facilitate Formal Verification of RTL. *arXiv preprint arXiv:2309.09437* (2023).
- [16] David Sheridan, Lingyi Liu, Hyungsul Kim, and Shobha Vasudevan. 2014. A coverage guided mining approach for automatic generation of succinct assertions. In *2014 27th International Conference on VLSI Design and 2014 13th International Conference on Embedded Systems*. IEEE, 68–73.
- [17] Chuyue Sun, Christopher Hahn, and Caroline Trippel. 2023. Towards Improving Verification Productivity with Circuit-Aware Translation of Natural Language to SystemVerilog Assertions. In *Design Automation Conference Young Fellows Workshop (DAV)*. OpenReview.
- [18] Shobha Vasudevan, David Sheridan, Sanjay Patel, David Tchong, Bill Tuohy, and Daniel Johnson. 2010. Goldmine: Automatic assertion generation using data mining and static analysis. In *2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*. IEEE, 626–629.
- [19] Yonghao Wang, Jiaxin Zhou, Yang Yin, Hongqin Lyu, Zhiteng Chao, Wenchao Ding, Jing Ye, Tiancheng Wang, and Huawei Li. 2026. CoverAssert: Iterative LLM-Based Assertion Generation Using Syntax-Semantic Representations for Functional Coverage-Guided Verification. *arXiv preprint arXiv:2602.15388* (2026).
- [20] Zhiyuan Yan, Wenji Fang, Mengming Li, Min Li, Shang Liu, Zhiyao Xie, and Hongce Zhang. 2025. AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs. In *30th Asia and South Pacific Design Automation Conference*. 614–621.
- [21] Zhiyuan Yan and Hongce Zhang. 2024. Word-Level Augmentation of Formal Proof by Learning from Simulation Traces. In *2024 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–8. <https://doi.org/10.1145/3676536.3676686>
- [22] Leiqi Ye, Yixuan Li, Guy Frankel, Jianyi Cheng, and Elizabeth Polgreen. 2025. Unlocking hardware verification with oracle guided synthesis. In *The 25th Conference on Formal Methods in Computer-Aided Design*. TU Wien Academic Press, 235–245.
- [23] YosysHQ. 2026. PicoRV32: A Size-Optimized RISC-V CPU. <https://github.com/YosysHQ/picorv32>. Accessed: 2026-04-14.
- [24] YosysHQ. 2026. SymbiYosys (SBY) Documentation. <https://symbiyosys.readthedocs.io/en/latest/>. Accessed: 2026-04-12.
- [25] Ziyue Zheng, Zhiyuan Yan, Xiangchen Meng, Guangyu Hu, Hongce Zhang, and Yangdi Lyu. 2025. Hot-FV: A Semi-Formal Test Generation Framework for RTL Functional Coverage Using Warm Starting States. In *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. IEEE, 298–305.